

Web Privacy TP – Browser Fingerprinting

Mission: Follow Alice

Zineb Elansari

October 2, 2026

Introduction

Cookies can be deleted or blocked, but a browser still reveals many details about its environment. In this lab, I built a small local fingerprinting prototype with Flask and JavaScript. It collects passive features (HTTP headers), active features (browser APIs) and behavioural features (typing). I also tested several protection mechanisms.

1 Part 1 – Investigating the HTTP Request

1.1 Task 0: HTTP headers seen in the browser

Figures 1.1 and 1.1 show the request headers sent by the browser (Developer Tools → Network → Headers) when visiting a website.

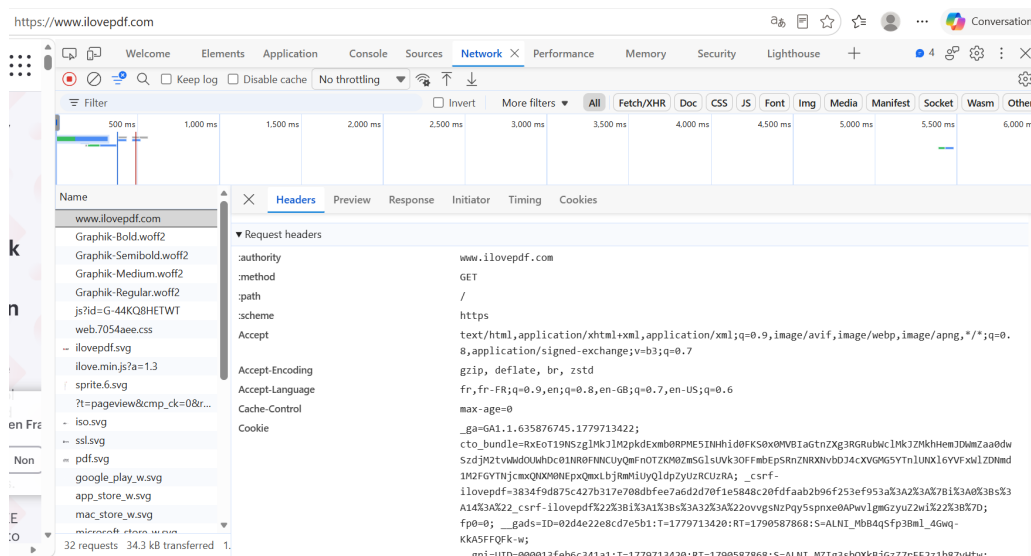


Figure 1: Request headers in Developer Tools (1/2).

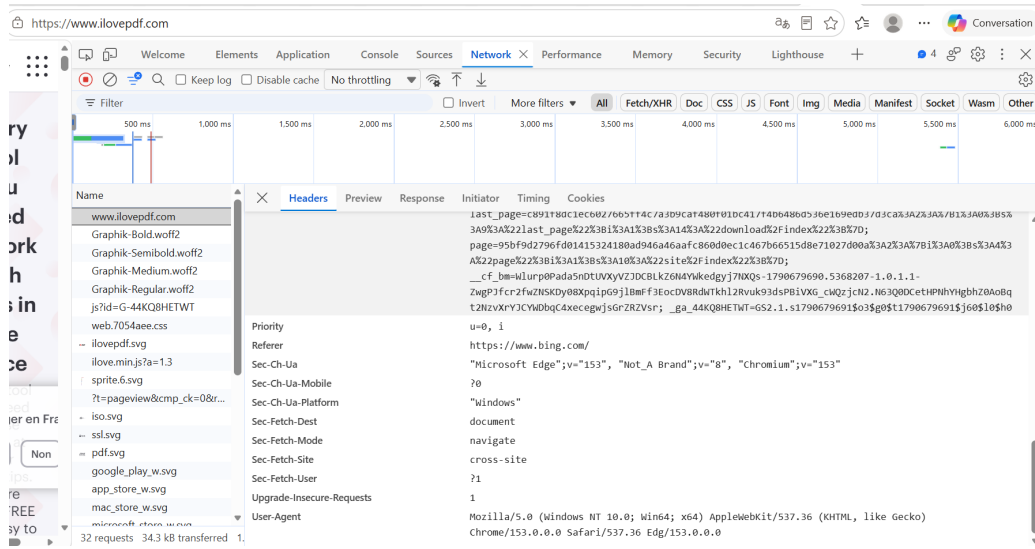


Figure 2: Request headers in Developer Tools (2/2).

Header	What it reveals	Fingerprinting value
User-Agent	Browser name/version, OS, architecture	High
Accept-Language	Preferred languages and their order	Medium-High
Accept	Content formats supported (AVIF, WebP, ...)	Medium
Accept-Encoding	Supported compressions (gzip, br, zstd)	Low-Medium (depends on version)
Sec-CH-UA	Browser brands and major versions	Medium
Sec-CH-UA-Platform	Operating system	Medium
Sec-CH-UA-Mobile	Mobile or desktop device	Low
Sec-Fetch-Site / Mode / Dest	Navigation context (cross-site, navigate, document)	Low
Referer	Previous page (here bing.com)	Context / tracking
Cookie	Identifiers stored by the site (_ga, _csrf, ...)	High (stateful tracking)
Cache-Control, Upgrade-Insecure-Requests, Priority	Cache and security behaviour	Low

Table 1: Request headers and their contribution to fingerprinting.

1.2 Task 1: Passive collector in Flask

In `app.py` I added, besides `User-Agent` and `Accept-Language`, the headers `Accept`, `Accept-Encoding`, `Sec-CH-UA`, `Sec-CH-UA-Mobile`, `Sec-CH-UA-Platform`, `Sec-Fetch-Site`, `Sec-Fetch-Mode` and `Sec-Fetch-Dest`. I then visited the page with Microsoft Edge, Google Chrome and Microsoft Edge in private mode.

```

===== NEW VISIT =====
IP address      : 127.0.0.1
HTTP method     : GET
User-Agent      : Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/153.0.0.0 Safari/537.36 E
dg/153.0.0.0
Accept-Language : fr,fr-FR;q=0.9,en;q=0.8,en-GB;q=0.7,en-US;q=0.6
Accept          : text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/sign
ed-exchange;v=b3;q=0.7
Accept-Encoding : gzip, deflate, br, zstd
Sec-CH-UA      : "Microsoft Edge";v="153", "Not_A Brand";v="8", "Chromium";v="153"
Sec-CH-UA-Mobile : ?0
Sec-CH-UA-Platform: "Windows"
Sec-Fetch-Site  : none
Sec-Fetch-Mode  : navigate
Sec-Fetch-Dest  : document
=====

```

Figure 3: Flask console – Microsoft Edge (normal).

```

===== NEW VISIT =====
IP address      : 127.0.0.1
HTTP method     : GET
User-Agent      : Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/154.0.0.0 Safari/537.36
Accept-Language : fr-FR,fr;q=0.9,en-US;q=0.8,en;q=0.7
Accept          : text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/sign
ed-exchange;v=b3;q=0.7
Accept-Encoding : gzip, deflate, br, zstd
Sec-CH-UA      : "Chromium";v="154", "Google Chrome";v="154", "Not A(Brand";v="99"
Sec-CH-UA-Mobile : ?0
Sec-CH-UA-Platform: "Windows"
Sec-Fetch-Site  : none
Sec-Fetch-Mode  : navigate
Sec-Fetch-Dest  : document
=====

```

Figure 4: Flask console – Google Chrome.

```

===== NEW VISIT =====
IP address      : 127.0.0.1
HTTP method     : GET
User-Agent      : Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/153.0.0.0 Safari/537.36 E
dg/153.0.0.0
Accept-Language : fr,fr-FR;q=0.9,en;q=0.8,en-GB;q=0.7,en-US;q=0.6
Accept          : text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/sign
ed-exchange;v=b3;q=0.7
Accept-Encoding : gzip, deflate, br, zstd
Sec-CH-UA      : "Microsoft Edge";v="153", "Not_A Brand";v="8", "Chromium";v="153"
Sec-CH-UA-Mobile : ?0
Sec-CH-UA-Platform: "Windows"
Sec-Fetch-Site  : none
Sec-Fetch-Mode  : navigate
Sec-Fetch-Dest  : document
=====

```

Figure 5: Flask console – Microsoft Edge (private mode).

Header	Edge	Chrome	Edge private
User-Agent	... Chrome/153 ... Edg/153	... Chrome/154 (no Edg)	Same as Edge
Accept-Language	fr,fr-FR;q=0.9,en;q=0.8,en-GB;q=0.7,en-US;q=0.6	fr-FR,fr;q=0.9,en-US;q=0.8,en;q=0.7	Same as Edge
Sec-CH-UA	Microsoft Edge 153, Chromium 153	Chromium 154, Google Chrome 154	Same as Edge
Platform / Mobile	"Windows" / ?0	"Windows" / ?0	Same
Accept, Accept-Encoding	identical	identical	identical
Sec-Fetch-*	none / navigate / document	same	same

Table 2: Comparison of passive values.

Analysis. The values that stay the same across browsers are the operating system (Windows, desktop), **Accept**, **Accept-Encoding** and the **Sec-Fetch-*** headers. The values that change are the **User-Agent**, **Sec-CH-UA** (brand and version) and the **Accept-Language** order, because Edge and Chrome have different versions and settings. Private mode does not change anything: Edge and Edge private send exactly the same headers. Private browsing only prevents local storage (cookies, history); it does not hide the browser’s environment from the server.

2 Part 2 – Active Feature Collection

2.1 Collected features

Using JavaScript, I collected six features, displayed them on the page, and sent them to Flask with a POST request to /collect.

Feature (API)	What it reveals	Why it helps distinguish browsers
navigator.language	Browser interface language	Differs by user/region; combined with the language list it is quite specific
Intl...timeZone	Time zone	Gives approximate location of the user
screen.width/height	Screen resolution	Many different resolutions exist
screen.colorDepth	Colour depth	Rarely different but adds entropy
navigator.hardwareConcurrency	Number of logical CPU cores	Reveals hardware class (here 16)
window.innerWidth/innerHeight	Browser window size	Varies with toolbars, zoom, extensions

Table 3: Active features.

2.2 Normal browser

```
===== ACTIVE FEATURES =====  
Browser language      : fr  
Time zone             : UTC  
Screen resolution    : 1280 x 720  
Color depth          : 24  
Logical CPU cores    : 16  
Browser window size: 1272 x 588  
=====
```

Figure 6: Flask console – active features received (normal browser).

Active Feature Collection

Collected Active Features

Browser language: fr

Time zone: UTC

Screen resolution: 1280 x 720

Color depth: 24

Logical CPU cores: 16

Browser window size: 1272 x 588

Figure 7: Webpage displaying the active features.

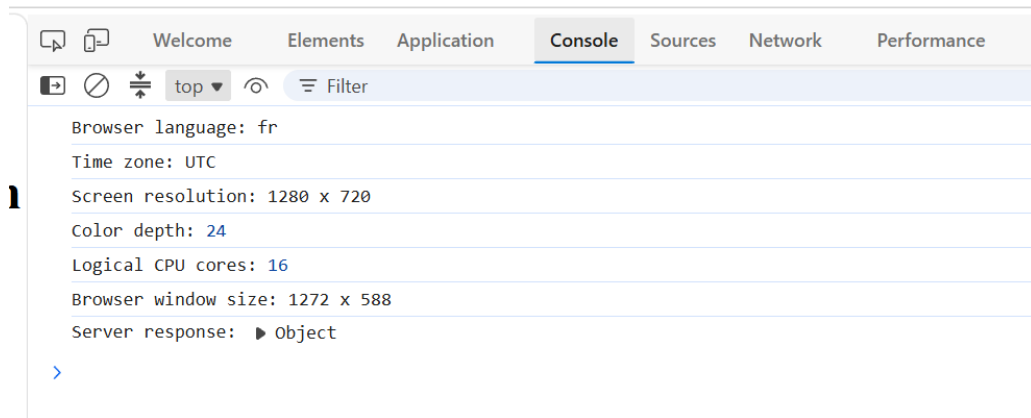


Figure 8: Developer Tools console showing the collected values and the server response.

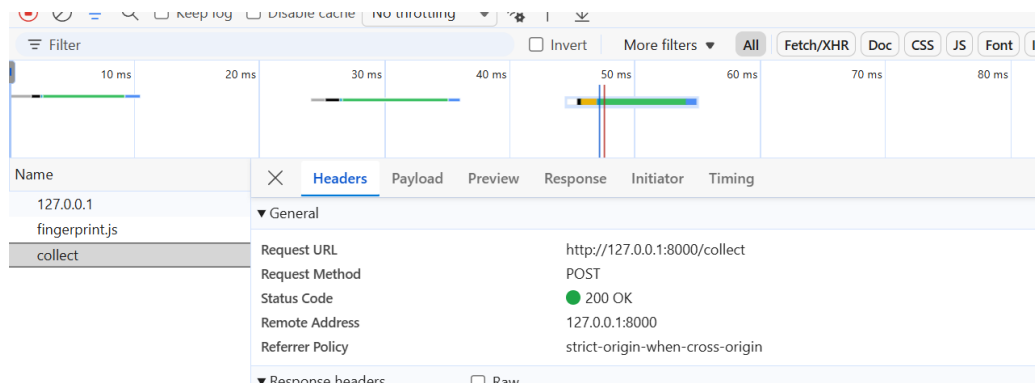
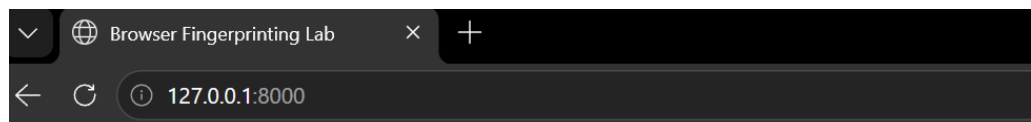


Figure 9: Network tab: POST `/collect` returns 200 OK.

2.3 Private-browsing mode



Active Feature Collection

Collected Active Features

Browser language: fr

Time zone: UTC

Screen resolution: 1280 x 720

Color depth: 24

Logical CPU cores: 16

Browser window size: 1272 x 588

Figure 10: Active features in private-browsing mode.

In private mode the values are exactly the same as in the normal browser (fr, UTC, 1280×720 , 24, 16, 1272×588). Private mode therefore does not protect against active fingerprinting.

2.4 Cookie blocking

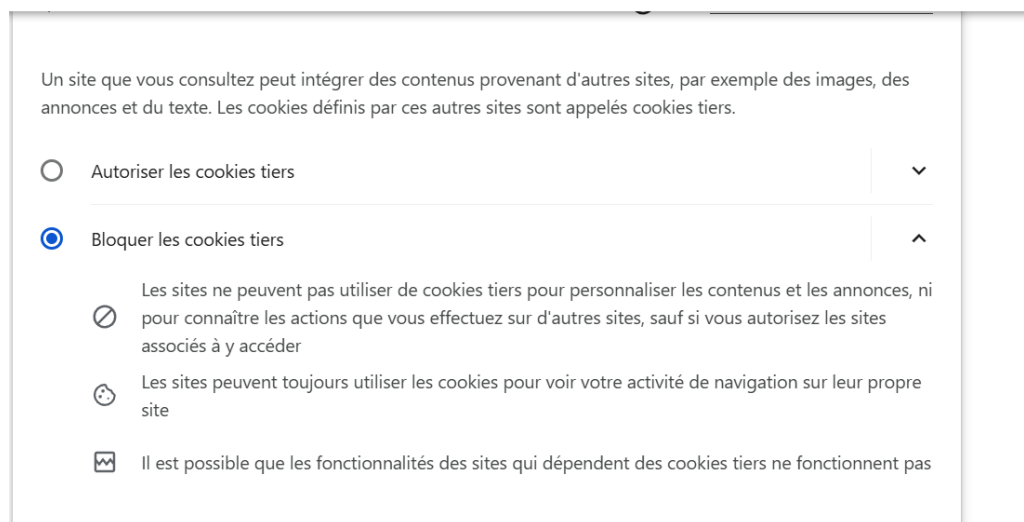


Figure 11: Third-party cookie blocking enabled in the browser settings.

Active Feature Collection

Collected Active Features

Browser language: fr-FR

Time zone: UTC

Screen resolution: 1280 x 720

Color depth: 24

Logical CPU cores: 16

Browser window size: 1280 x 529

Figure 12: Active features with cookie blocking enabled.

All features are still collected. Blocking cookies has no effect on fingerprinting, because the script does not store anything: it reads properties of the browser. The language appears as **fr-FR** and the window height as 529 here because this test was run in another browser/window (Chrome) than the Edge tests.

2.5 Privacy-preserving extension

I tested the extension *Fingerprint Protector* (v3.5.1) with the “Maximum” preset, showing 18/21 modules active.

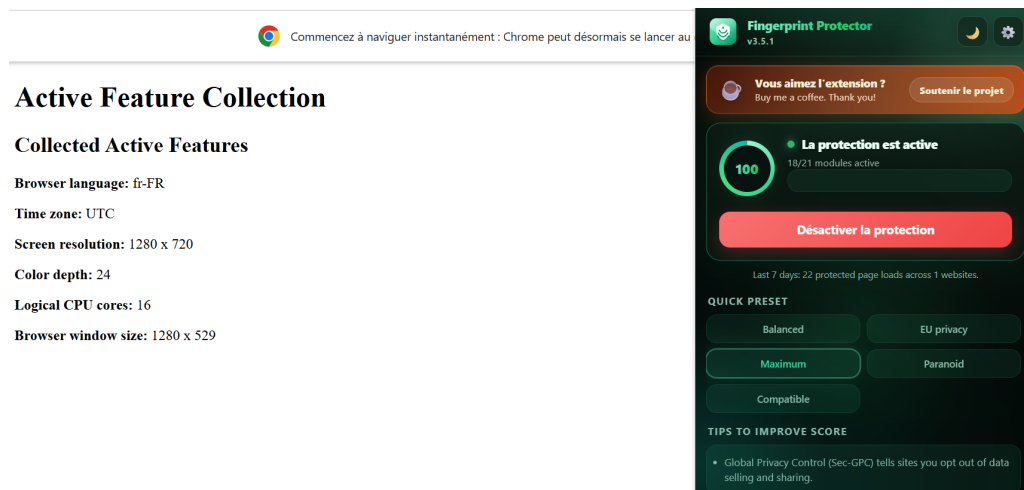


Figure 13: Fingerprint Protector extension active with the Maximum preset.

In my test, the six basic features were still returned by the page (language fr-FR, UTC, 1280×720 , 24, 16 cores, window 1280×529), so the extension did not visibly change these simple values. Its modules mainly target more advanced vectors (canvas, WebGL, audio, fonts, etc.), which were not part of my prototype.

2.6 Blocking the request URL

Using Developer Tools → Network → right-click on `fingerprint.js` → Block request URL, I blocked the fingerprinting script.

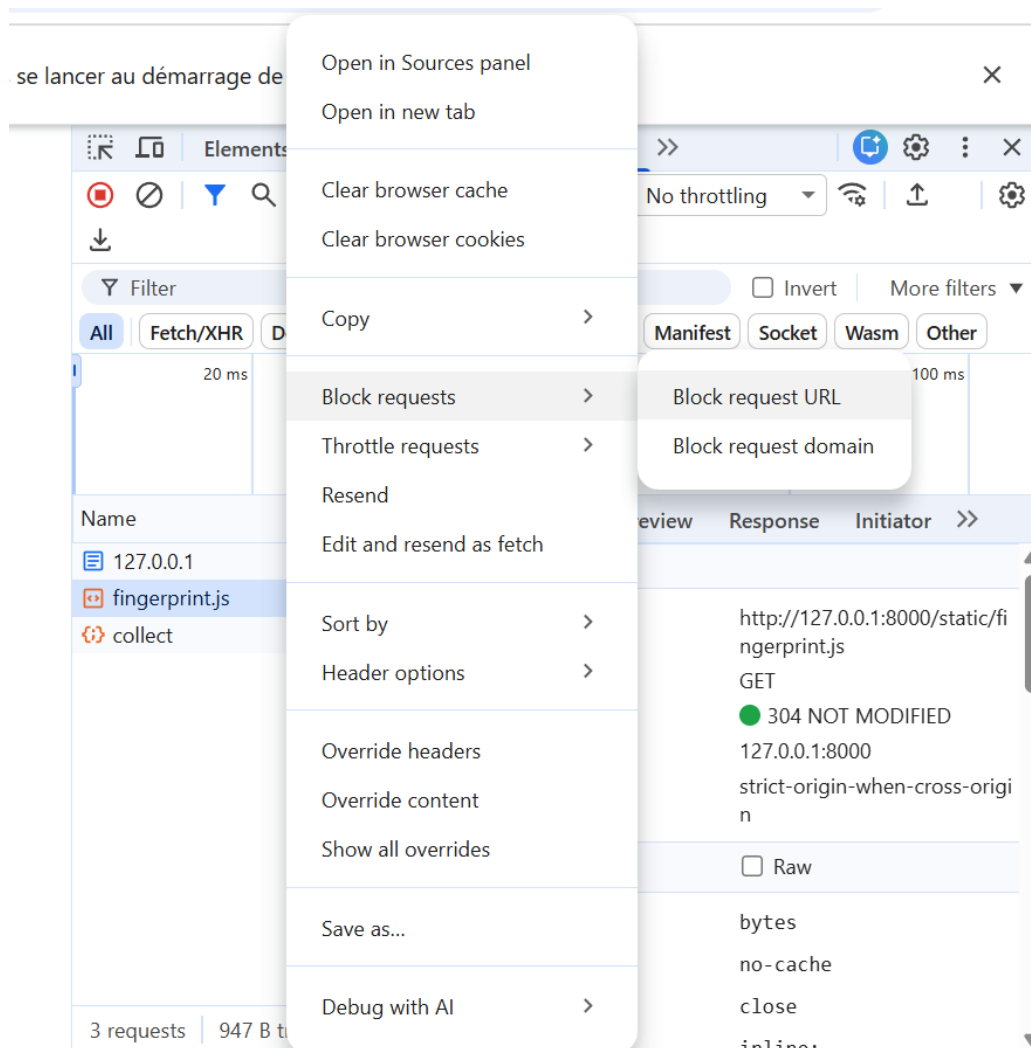
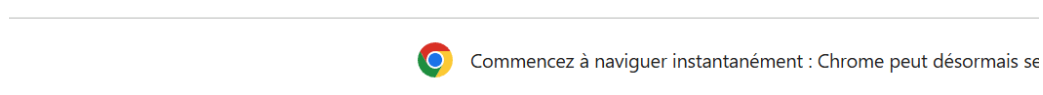


Figure 14: Blocking the request URL of `fingerprint.js`.



Active Feature Collection

Waiting for JavaScript...

Figure 15: Result: the page stays on “Waiting for JavaScript...”.

Without the script, no active feature is collected and nothing is sent to `/collect`.

2.7 Comparison and conclusion

Protection	Effect on features	Limitation
Private mode	None	Only removes local traces (cookies, history)
Cookie blocking	None	Fingerprinting uses no cookie
Fingerprint extension	No visible change on the basic features tested	Spoofed or randomised values can themselves be unusual; may break websites; coverage depends on modules
Blocking the script	Active collection fully stopped	Manual, specific to one script; passive HTTP headers are still sent

Table 4: Comparison of protections.

In this experiment, the most effective protection was blocking the fingerprinting script, since it stopped all active collection. In practice, a script blocker or a privacy-focused browser is more realistic, as a user cannot know every tracking script in advance. Even then, the passive headers (User-Agent, Accept-Language, ...) are still sent with each request, so no single measure removes fingerprinting completely.

3 Part 3 – The Typing Imposter

3.1 Typing speed formula

The target sentence has 57 characters. The typing speed is

$$\text{speed} = \frac{\text{number of characters}}{\text{typing time (s)}}$$

For example, $57/11.49 = 4.96$ characters/second.

3.2 Algorithm

1. Get the sentence, the input field, the restart button and the result area using `getElementById()`.
2. Initialise `startTime = null` and `corrections = 0`.
3. On `keydown`: if `startTime` is empty, set it to `performance.now()`; if `event.key === "Backspace"`, increase `corrections`.
4. On `input`: if `input.value === sentence`, compute the elapsed time $(\text{now} - \text{startTime})/1000$ and the speed.
5. Display the time, speed (with `toFixed(2)`) and corrections, then send them to `/collect` with `fetch()` and `JSON.stringify()`.
6. On `click` of the restart button: clear the input, the results and the variables.

3.3 Results

The Typing Imposter

Reproduce the following sentence exactly:

Cookies are gone, but Alice's rhythm still leaves traces!

's rhythm still leaves traces!

Restart Experiment

Results

Total typing time: 11.49 seconds

Typing speed: 4.96 characters/second

Corrections: 6

Figure 16: First run: slow typing with several corrections (a human-like behaviour).

The Typing Imposter

Reproduce the following sentence exactly:

Cookies are gone, but Alice's rhythm still leaves traces!

Cookies are gone, but Alice

Restart Experiment

Results

Total typing time: 2.06 seconds

Typing speed: 27.63 characters/second

Corrections: 0

Figure 17: Second run: very fast and without any correction.

The first run took 11.49 s (4.96 char/s) with 6 corrections. The second run took only 2.06 s (27.63 char/s) with 0 corrections. 27 characters per second is not realistic for manual typing, so this run looks like pasted or automated input: this is the “imposter”. It shows that behaviour can distinguish a real user from a script, but also that one user’s behaviour varies between sessions (4.96, 4.81, 9.83 char/s in my runs), so the result is only probabilistic.

3.4 Sending data to the server

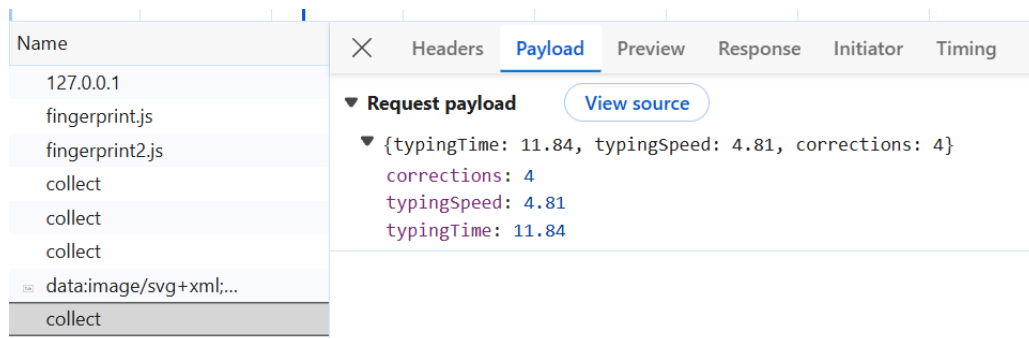


Figure 18: Request payload of POST /collect: typingTime 11.84, typingSpeed 4.81, corrections 4.

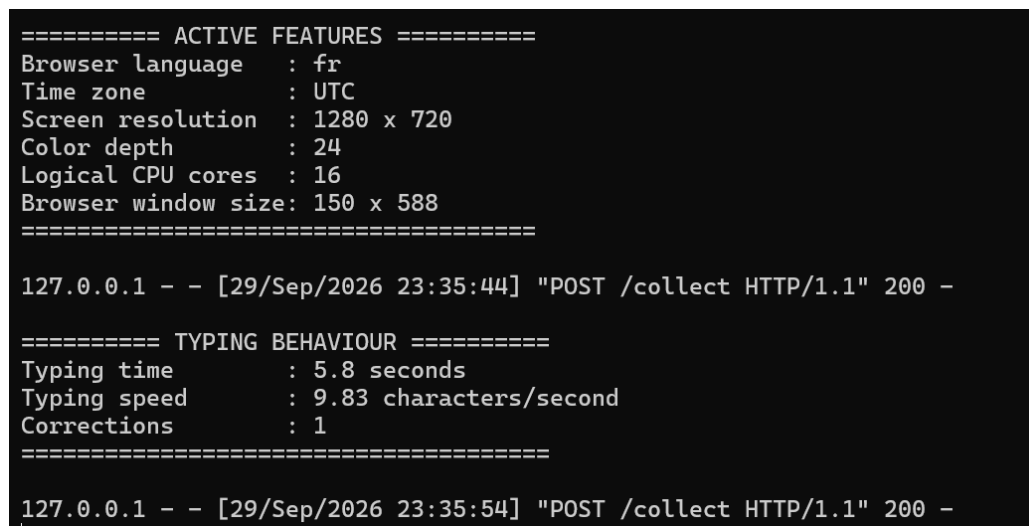


Figure 19: Flask console: active features and typing behaviour received (5.8s, 9.83 char/s, 1 correction).

The payload in Developer Tools matches the data printed by Flask, and the server answers 200.

4 Part 4 – Optional: Building the Fingerprint

4.1 Building the fingerprint

In a third script (`fingerprint3.js`), I combined the active features in a fixed order into one string (language | time zone | screen resolution | colour depth | CPU cores | window size). I hashed this string with SHA-256 using `crypto.subtle.digest()`, displayed the result on the page, and sent **only the hash** to the server.

```

===== ACTIVE FEATURES =====
Browser language : fr
Time zone       : UTC
Screen resolution : 1280 x 720
Color depth      : 24
Logical CPU cores : 16
Browser window size: 1272 x 588
=====

127.0.0.1 - - [29/Sep/2026 23:50:56] "POST /collect HTTP/1.1" 200 -

===== FINGERPRINT =====
SHA-256 fingerprint: 470ca6b82dc598b0c5ed287ebf64deb3b6b6570940a060e7f176f051ce4a891
=====

```

Figure 20: Flask console: the active features and the SHA-256 fingerprint received.

Screen resolution: 1280 x 720

Color depth: 24

Logical CPU cores: 16

Browser window size: 1272 x 588

The Typing Imposter

Reproduce the following sentence exactly:

Cookies are gone, but Alice's rhythm still leaves traces!

Part 4 — Fingerprint Identifier

The collected browser features are combined and transformed into a SHA-256 fingerprint.

Browser Fingerprint

SHA-256 fingerprint: 470ca6b82dc598b0c5ed287ebf64deb3b6b6570940a060e7f176f051ce4a891

Figure 21: The fingerprint displayed on the webpage; it is identical to the one received by the server.

4.2 Stability test

Generating the fingerprint several times in the same browser, with the same window, gives the same value (470ca6b8...4a891), because the same input always gives the same hash. I then made the browser window about half as wide (1272 × 588 became 630 × 587).

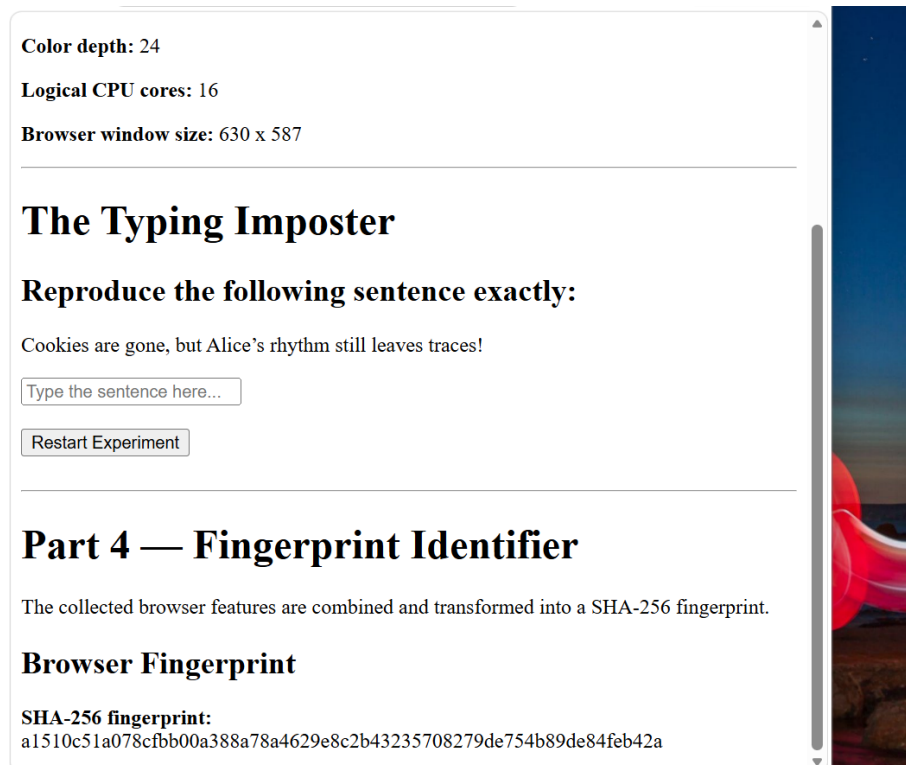


Figure 22: Fingerprint after resizing the window (a1510c51...eb42a).

The fingerprint changed completely, although only the window size was different. This is the avalanche effect of SHA-256: a small change in the input produces a totally different hash. It also shows that the window size is an unstable feature. A real tracker would remove it, or other volatile features, to keep a stable identifier for the same browser.

4.3 Sending only the hash

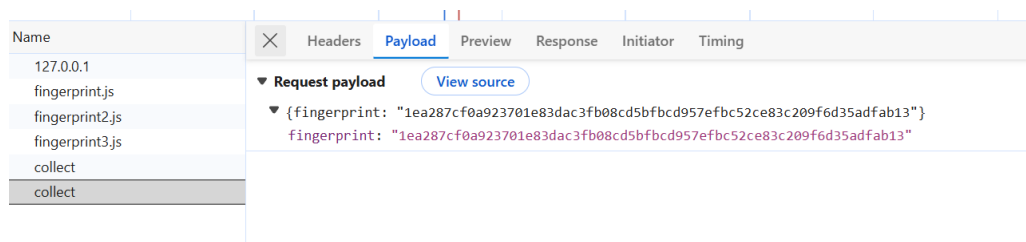


Figure 23: Request payload of POST /collect: only the **fingerprint** field is sent.

The payload contains only the hash, not the raw features. The server can compare identifiers easily, and the user cannot see the language, screen or hardware information in the request. This is why fingerprinting is difficult to notice. (The hash in this screenshot differs from the previous ones because it comes from a different run/browser with different feature values.)

4.4 Comparison with colleagues

Two browsers with exactly the same features would produce the same hash. Browsers with different languages, screens or hardware produce different hashes. With only six features, collisions between users with similar machines are likely, so a real fingerprint uses many more features (canvas, WebGL, fonts, etc.) to be unique.

Conclusion

Passive headers and active JavaScript features can be combined to recognise a browser without any cookie. Private mode and cookie blocking do not stop this, while blocking the fingerprinting script does. Typing behaviour adds a further layer, but it is variable and therefore probabilistic. Fingerprinting is hard for ordinary users to notice, so defences must target the collection itself rather than stored identifiers.